02 - First model comparison

October 19, 2026

1 Introduction

This document presents the preliminary comparison of models for classifying music tracks by their genre. This first analysis has been run on the `FMA_small` dataset [1]. The main code used to run this analysis is `compare_models.py`, along as `feature_groups.py`. This code compare different classes of classifiers and rank their performances.

2 The dataset

The `FMA_small` dataset is presented in [1]. It contains 8000 audio clips limited to 30 seconds (extracted from the middle of the tracks). Each audio clip is associated to a single top genre. The 8 different top genres are evenly distributed among the 8000 audio clips.

The dataset is split into a training set, which contains 6400 audio clips, a validation test and a test set which both contain 800 clips each. The genres are evenly distributed in the 3 subsets. The validation and the test set both are built in a way that their is no artist leakage in between the sets: each artist is bound to a given set and no artist is present over more than 1 set. This allows to train models on a given set and to test it on a different set with no overlapping artist. So the prediction is always done on unknown artists.

3 The feature set

The features used for this analysis are different from the features given in the `FMA` dataset. A whole feature set has been computed for this analysis. The code used to compute these features is `do_features.py`.

In this feature set, there are 27 different kinds of "statistical" features. The statistical features are features for which 9 statistical quantities are computed: the mean, the median, the standard deviation, the skew, the kurtosis, the minimum, the maximum, the 10th and the 90th percentiles. For some of these 27 kinds of statistical features, a set of statistical features is given. For instance, the `MFCC` feature comes in a set of 20 statistical features, which means there are actually $20 \times 9 = 180$ features for the `MFCC` feature. The table 1 lists all statistical features and their respective quantity.

In addition to these statistical features, there are two scalar features: tempo and beat_rate. This brings the total to 1397 features. The computational time analysis of the feature set is presented in another document, named "Computational time analysis". It is present in the analysis directory on this repository.

Table 1: The list of all statistical features and their quantity.

Feature	Quantity
chroma_stft	12
chroma_cqt	12
chroma_cens	12
delta_chroma_stft	12
delta_chroma_cqt	12
tonnetz_cqt	6
tonnetz_cens	6
mfcc	20
delta_mfcc	20
delta2_mfcc	20
rms	1
zcr	1
spectral_centroid	1
spectral_bandwidth	1
spectral_contrast	7
spectral_flatness	1
spectral_flux	1
spectral_rolloff_50	1
spectral_rolloff_85	1
spectral_rolloff_95	1
harmonic_rms	1
percussive_rms	1
harmonic_ratio	1
percussive_ratio	1
onset_strength	1
tempogram_mean	1
beat_interval	1

4 Model selection

4.1 The models

The selection is done among different classes of models. The classes of models which have been tested are: logistic regression, K-nearest neighbors (KNN), support vector machine (SVM), random forest, extra trees, hist-gradient-boosting (HGB) and multilayer perceptron (MLP). All coming from the `sci-kit learn` library[2]. For each class of models, a grid of parameters has been investigated. Here is a list of the parameters which have been explored:

- logistic regression: `C`,
- KNN: `n-neighbors`, `weights`, `p`,
- SVM: `C`, `gamma`,
- random forest: `max_features`, `min_samples_leaf`, `n_estimators`, `class_weight`,
- extra trees: `max_features`, `min_samples_leaf`, `n_estimators`, `class_weight`,
- HGB: `learning_rate`, `max_leaf_nodes`, `min_samples_leaf`, `l2_regularization`,
- MLP: `hidden_layer_sizes`, `alpha`.

4.2 The training

The training is done using cross-validation with the `StratifiedGroupKFold` method from the `sci-kit learn` package. This method creates stratified folds with non-overlapping groups. The non-overlapping group chosen here are the artists. In total, 5 folds were used to do the training of the models. It means that each model is trained on 4/5 of the data and tested on the remaining 1/5. This remaining 1/5 has no overlapping artist with the training set, so the prediction is done on unknown artists. Models are trained 5 times and at the end a prediction is done for all the data. This prediction is used to compare models. The dataset used to do the cross-validation is the training set and the validation set combined. Which means the models are trained on 7200 tracks. The averaged macro F1 score over the 5 cross-validations is used to rank and select the models.

5 Results

Among all the models which have been tested, 3 different classes of models have been selected as promising. Here are the selected models (called best models in the following) with their parameters:

- SVM:
  - `C` = 1.3,
  - `gamma` = 0.000537,
- HGB:
  - `learning_rate` = 0.1,
  - `max_leaf_nodes` = 31,
  - `min_samples_leaf` = 10,

- `l2_regularization` $= 1$,

- MLP:
  - `hidden_layer_sizes` $= (64, 32)$,
  - `alpha` $= 10$.

The table 2 sums up the performances of each model. It gives the macro F1 score measured on the 5 cross-validation folds (out-of-fold) and then the macro F1 score measured on the test set. There is a significant difference in the F1 score between the out of fold measurement and the one made on the test set.

Table 2: The performances of each model. OOF stands for out-of-fold and TEST designates the performances measured on the test set.

	SVM	HGB	MLP
macro F1 OOF	0.554 ± 0.015	0.561 ± 0.013	0.551 ± 0.017
macro F1 TEST	0.476	0.490	0.494

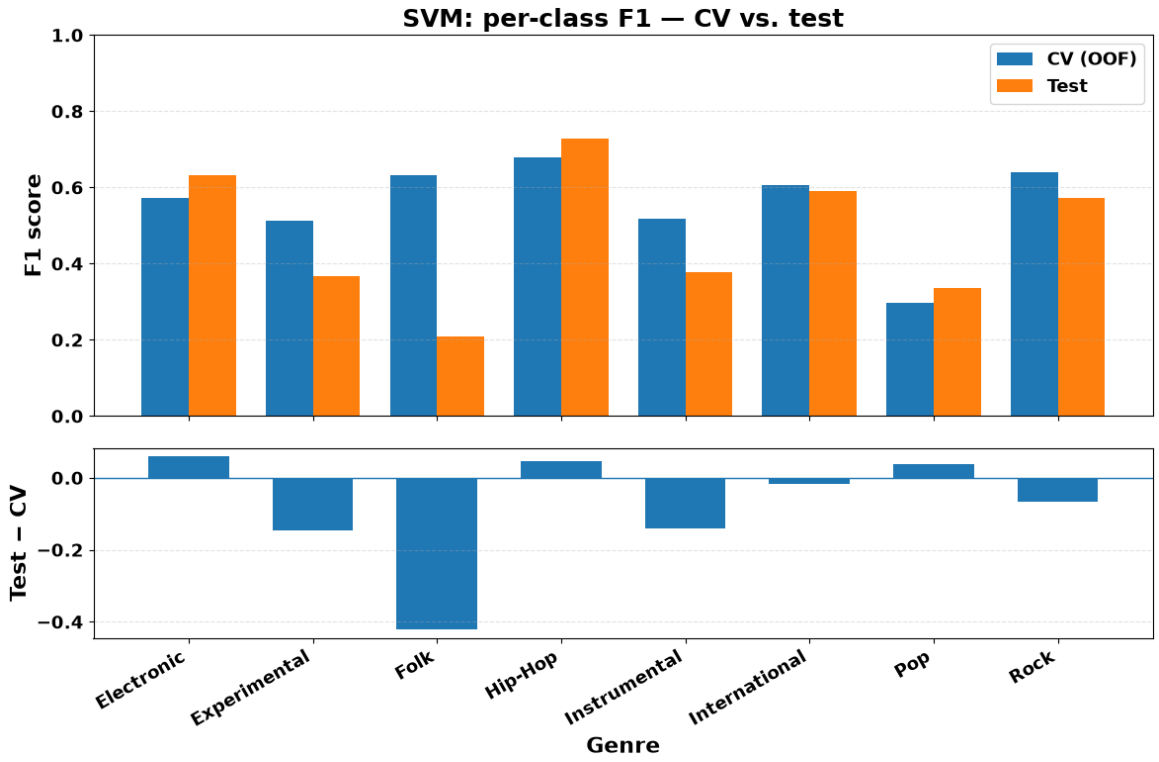


Figure 1: The F1 scores for out of fold prediction and test prediction for the best SVM model.

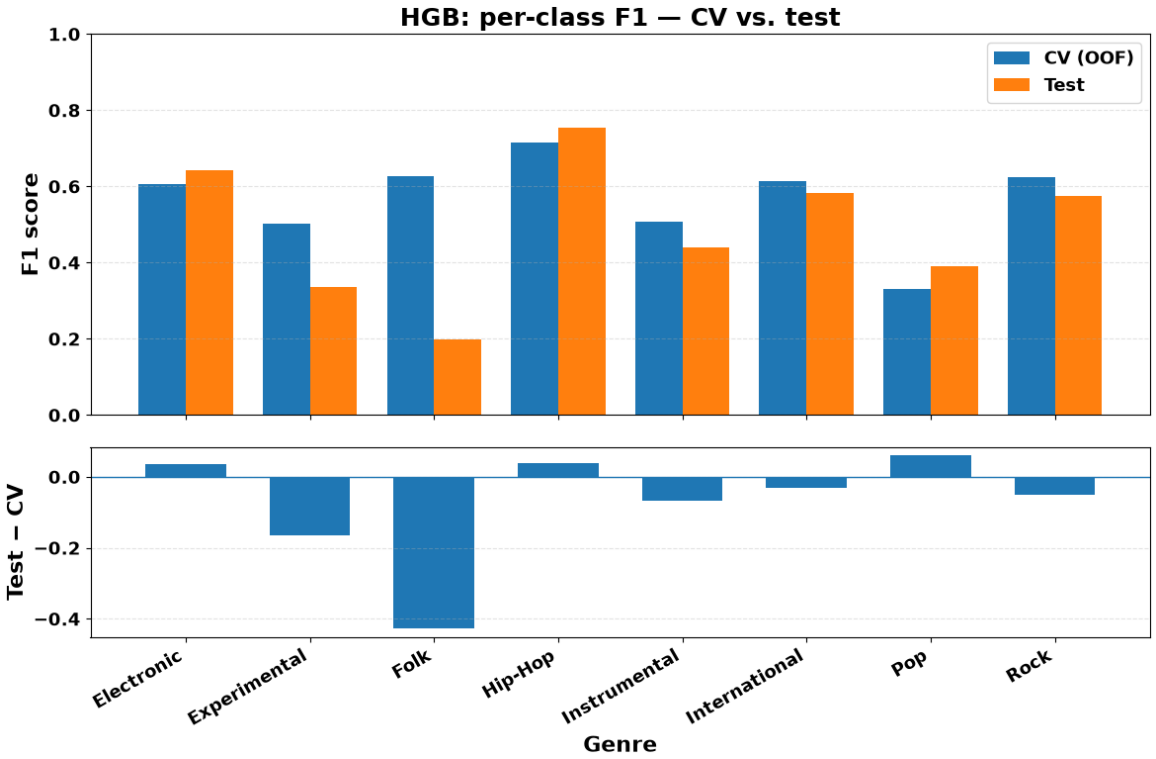


Figure 2: The F1 scores for out of fold prediction and test prediction for the best HGB model.

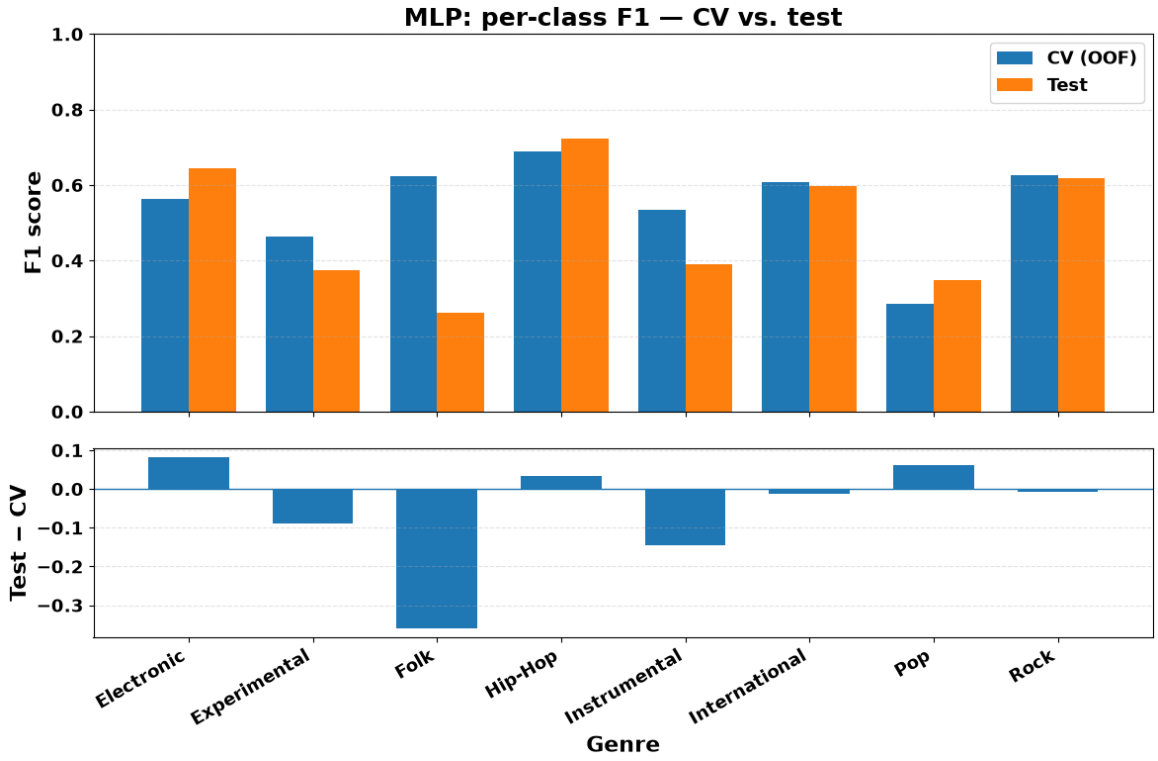


Figure 3: The F1 scores for out of fold prediction and test prediction for the best MLP model.

This difference on the macro F1 score has been investigated. The figures 1, 2 and 3 compare the F1 scores for the out-of-fold prediction and for the test prediction, for all the 8 genres. These figures show that the drop of performance is mainly due to a drop of performance to classify the Folk genre. The F1 score drops from 0.631 to 0.209 for SVM, from 0.627 to 0.199 for HGB and from 0.623 to 0.263 for MLP for this genre. There is also a significant drop of performance for the Experimental and Instrumental genres.

A model linked cause can be excluded since this behavior is observed for the 3 model classes. This means the difference comes from the data. So something specific to Folk (and partly Experimental and Instrumental) differs between the cross-validations and the test set. Since the genres are evenly distributed on all the dataset, the set used for the cross-validations (training + validation set) has the same genre distribution has the test set. So this difference in F1 scores is not coming from a difference on the genre distributions.

The figures 4 and 5 present the confusion matrices for the best HGB model, measured on the cross-validations and on the test set respectively. The biggest effect observable is a significant increase of Folk tracks classified as Experimental ones (False Negative) and a significant increase of Instrumental tracks classified as Folk ones (False Positive). This effect is seen on all best models. The effect is ranging from 0.11 to 0.23 points for the False Negative and from 0.15 to 0.29 points for the False Positive on the 3 models.

The difference on the macro F1 score can simply come from a test set, and especially the Folk tracks, which is quite different from the training set. Indeed, the test set Folk tracks are dominated by 1 artist: 91% of the tracks come from the same artist. And 68% of Hip-Hop tracks and 50% of Instrumental tracks come from the same artist in the test set. If this Folk artist is very different than the Folk artists the models have been trained on, it is not surprising the models perform worst on the test set. It is also possible that the audio recording quality is not a good as the quality on the training set, which would make even harder the prediction of the test set.

Because of these various elements, it is assumed here that the difference in macro F1 score on the training and on the test set is coming from a not representative enough test set. The assumption will be investigated in the next analysis with more data by going to a bigger dataset like `FMA_medium` or `FMA_large`.

6 Conclusion

Different classes of models have been trained on the `FMA_small` dataset. At the end of this analysis, 3 classes of models have been selected: SVM, HGB and MLP. The best models parameters are given in section 5. There is a significant difference between the macro F1 score on the cross-validations and on the test set. This difference is assumed to come from a test set which is not representative enough. Indeed, the test set Folk genre is dominated at 91% by 1 artist, and the F1 score for this genre classification drops by a $0.36 - 0.43$ points range for the 3 models.

Since this difference is assumed to be linked to a lack or data, the best model is chosen among the results on the cross-validations. And so the HGB model is the one that performs the best.

The next step is to increase the dataset size, by going to `FMA_medium` or `FMA_large`. But before that, the feature set has to be investigated to see if some of the features can be removed without degrading the performances. This will save time when computing the features on a big dataset. The plan is to first study the correlations among the features, and then perform an ablation analysis with the best model to conclude on the feature set.

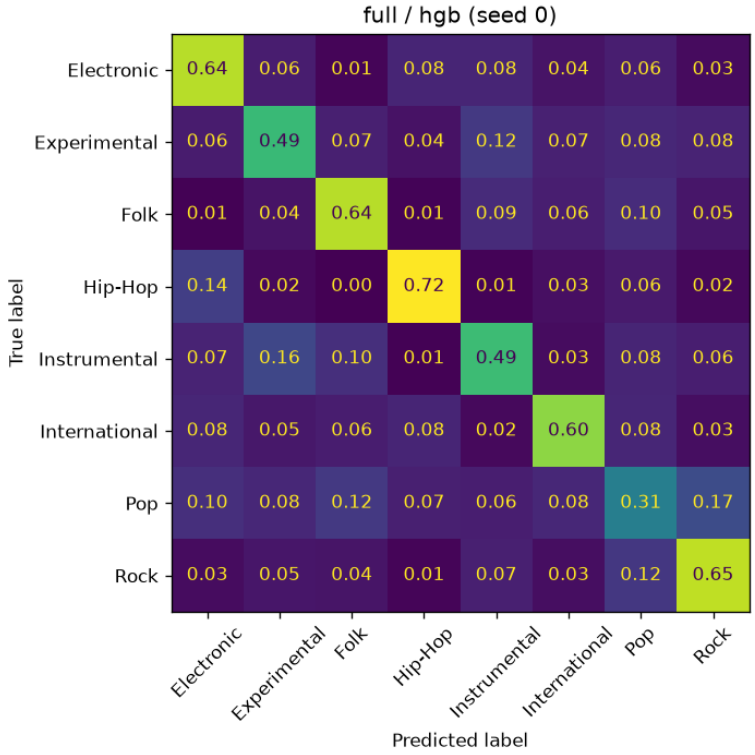


Figure 4: The confusion matrix for the best HGB model measured on the CV.

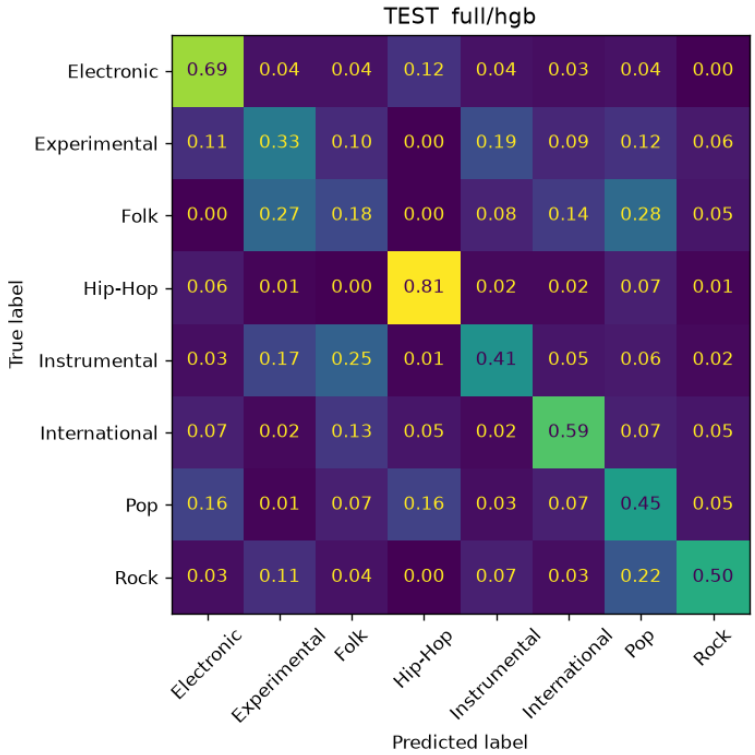


Figure 5: The confusion matrix for the best HGB model measured on the test set.

References

[1] Michaël Defferrard et al. "FMA: A Dataset for Music Analysis". In: *18th International Society for Music Information Retrieval Conference (ISMIR)*. 2017. arXiv: 1612.01840. URL: https://arxiv.org/abs/1612.01840.

[2] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.